

LAPORAN PRAKTIKUM STRUKTUR DATA
IMPLEMENTASI DAN TRAVERSAL POHON BINER



OLEH :

JOVANTRI IMMANUEL GULO

NIM 2411532014

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. Ir. Wahyudi, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

A. Pendahuluan

Di antara banyaknya bentuk struktur data, terdapat struktur data non-linear seperti pohon (tree) dan graf (graph) punya peran yang penting dalam menyelesaikan berbagai macam masalah komputasi seperti pencarian, pengurutan (sorting), bahkan mencari hubungan atau representasi antar data. Pohon biner (binary tree) merupakan salah satu bentuk struktur data yang berbentuk pohon, yang di mana pada pohon biner hanya bisa memiliki anak maksimal sebanyak 2 simpul (node). Dalam memahami cara kerja pohon biner, diperlukan eksplorasi terhadap berbagai metode traversal pada pohon biner, di antaranya adalah preorder, inorder, dan postorder. Sedangkan, graf merupakan salah satu bentuk struktur data yang digunakan untuk merepresentasikan hubungan antara objek yang satu dengan objek lainnya. Untuk memahami cara kerja pohon graf, diperlukan untuk memahami macam-macam teknik traversal graf di antaranya adalah Depth First Search (DFS) dan Breadth First Search (BFS). Kedua algoritma graf tersebut seringkali digunakan dalam melakukan pencarian jalur, sistem navigasi, dan aplikasi lain dalam kehidupan nyata.

B. Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Mampu memahami cara mengimplementasikan pohon biner
2. Mampu memahami cara mengimplementasikan pohon graf
3. Mampu mengimplementasikan traversal preorder, inorder, dan postorder
4. Mampu mengimplementasikan traversal depth first search (dfs) dan breadth first search (bfs)

C. Langkah – langkah Pengerjaan

Berikut adalah langkah-langkah dalam pengerjaan praktikum kali ini:

1. Buat file class dengan nama class **Node**
2. Deklarasikan class Node dan konstruktornya, serta deklarasikan nama-nama variabel yang diperlukan pada class tersebut, seperti int data, Node left, dan Node right.

```
3 public class Node {
4     int data;
5     Node left;
6     Node right;
7
8     public Node(int data) {
9         this.data = data;
10        left = null;
11        right = null;
12    }
```

3. Membuat method setLeft, setRight, getLeft, getRight, getData, dan setData dengan melakukan return pada logika method masing-masing.

```

14 public void setLeft(Node node) {
15     if (left == null)
16         left = node;
17 }
18
19 public void setRight(Node node) {
20     if (right == null)
21         right = node;
22 }
23
24 public Node getLeft() {
25     return left;
26 }
27
28 public Node getRight() {
29     return right;
30 }
31
32 public int getData() {
33     return data;
34 }
35
36 public void setData(int data) {
37     this.data = data;
38 }

```

4. Membuat method printPreOrder, yang kerjanya mencetak node sekarang, dan rekursif ke kiri lalu rekursif ke kanan

```

40 void printPreorder(Node node) {
41     if (node == null)
42         return;
43     System.out.print(node.data + " ");
44     printPreorder(node.left);
45     printPreorder(node.right);
46 }

```

5. Method printPostOrder, yang kerjanya rekursif ke kiri terlebih dahulu, rekursif ke kanan, lalu mencetak node yang sekarang.

```

48 void printPostorder(Node node) {
49     if (node == null)
50         return;
51     printPostorder(node.left);
52     printPostorder(node.right);
53     System.out.print(node.data + " ");
54 }

```

6. Method printInOrder, yang cara kerjanya rekursif ke kiri terlebih dahulu, kemudian cetak node yang sekarang, lalu rekursif ke kanan.

```

56 void printInorder(Node node) {
57     if (node == null)
58         return;
59     printInorder(node.left);
60     System.out.print(node.data + " ");
61     printInorder(node.right);
62 }

```

7. Memanggil method print dengan parameter awal kosong, kemudian membuat method print yang mencetak tampilan pohon secara visual dalam format/bentuk cabang.

```
64 public String print() {
65     return this.print("", true, "");
66 }
67
68 public String print(String prefix, boolean isTail, String sb) {
69     if (right != null) {
70         right.print(prefix + (isTail ? "|   " : "   "), false, sb);
71     }
72     System.out.println(prefix + (isTail ? "└─ " : "└─ ") + data);
73     if (left != null) {
74         left.print(prefix + (isTail ? "   " : "|   "), true, sb);
75     }
76     return sb;
77 }
78 }
```

8. Pembuatan class Node sudah selesai, sekarang perlu membuat class dengan nama **BTree**
9. Deklarasikan terlebih dahulu nama package, class dan juga konstruktor untuk kelas BTree tersebut

```
1 package pekan9;
2 // Jovantri Immanuel Gulo - 2411532014
3 public class BTree {
4     private Node root;
5     private Node currentNode;
6
7     public BTree() {
8         root = null;
9     }
10 }
```

10. Buat method search dengan tipe data boolean untuk melakukan search dengan keyword root dan data

```
11 public boolean search(int data) {
12     return search(root, data);
13 }
14
15 private boolean search(Node node, int data) {
16     if (node.getData() == data)
17         return true;
18     if (node.getLeft() != null)
19         if (search(node.getLeft(), data))
20             return true;
21     if (node.getRight() != null)
22         if (search(node.getRight(), data))
23             return true;
24     return false;
25 }
```

11. Class ini juga menyediakan tiga metode traversal yaitu printInorder(), printPreorder(), dan printPostorder(), yang akan mencetak isi pohon dengan urutan berbeda sesuai jenis traversalnya. Ketiga metode ini menggunakan metode traversal yang sudah didefinisikan dalam class Node.

```

27 public void printInorder() {
28     root.printInorder(root);
29 }
30
31 public void printPreorder() {
32     root.printPreorder(root);
33 }
34
35 public void printPostorder() {
36     root.printPostorder(root);
37 }
38
39 public Node getRoot() {
40     return root;
41 }

```

12. Method isEmpty() digunakan untuk mengecek apakah pohon masih kosong, selain itu, terdapat method countNodes() yang berguna untuk menghitung total jumlah simpul dalam pohon. Penghitungan ini dilakukan secara rekursif, yaitu dengan menambahkan simpul akar, jumlah simpul di anak kiri, dan jumlah simpul di anak kanan.

```

42 public boolean isEmpty() {
43     return root == null;
44 }
45
46 public int countNodes() {
47     return countNodes(root);
48 }

```

13. Terdapat juga method print() yang digunakan untuk menampilkan struktur pohon secara visual dalam bentuk garis bercabang, sehingga kita bisa melihat seperti apa bentuk pohon tersebut secara hierarkis.

```

61 public void print() {
62     root.print();
63 }

```

14. Untuk keperluan akses dan manipulasi data, disediakan juga method getter dan setter seperti getRoot(), getCurrent(), setCurrent(Node), dan setRoot(Node). Terakhir, method isEmpty() digunakan untuk mengecek apakah pohon masih kosong atau sudah memiliki data, dengan cara memeriksa apakah nilai root adalah null.

```

65 public Node getCurrent() {
66     return currentNode;
67 }
68
69 public void setCurrent(Node node) {
70     this.currentNode = node;
71 }
72
73 public void setRoot(Node root) {
74     this.root = root;
75 }

```

15. Buat file class baru dengan nama **TreeMain**

16. Deklarasikan package dan class TreeMain tersebut, buat objek pohon baru dan menampilkan bahwa jumlah simpul awalnya adalah nol

```

1 package pekan9;
2 // Jovantri Immanuel Gulo - 2411532014
3 public class TreeMain {
4     public static void main(String[] args) {
5         // Membuat Pohon
6         BTree tree = new BTree();
7         System.out.print("Jumlah simpul awal pohon: ");
8         System.out.println(tree.countNodes());

```

17. Membuat simpul dengan data 1 dan menetapkan sebagai root, lalu menampilkan jumlah simpul (harusnya 1).

```

11         Node root = new Node(1);
12
13         // Menjadikan simpul 1 sebagai root
14         tree.setRoot(root);
15         System.out.println("Jumlah simpul jika hanya ada root");
16         System.out.println(tree.countNodes());

```

18. Membuat simpul-simpul lainnya dan menyusunnya secara manual membentuk struktur pohon.

```

18         Node node2 = new Node(2);
19         Node node3 = new Node(3);
20         Node node4 = new Node(4);
21         Node node5 = new Node(5);
22         Node node6 = new Node(6);
23         Node node7 = new Node(7);
24
25         root.setLeft(node2);
26         node2.setLeft(node4);
27         node2.setRight(node5);
28         node3.setLeft(node6);
29         root.setRight(node3);
30         node3.setRight(node7);

```

19. Menyimpan root sebagai simpul yang sedang aktif, lalu menampilkannya.

```

33         tree.setCurrent(tree.getRoot());
34         System.out.println("Menampilkan simpul terakhir: ");
35         System.out.println(tree.getCurrent().getData());

```

20. Menampilkan jumlah simpul keseluruhan setelah pohon lengkap (harusnya 7).

```

37         System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
38         System.out.println(tree.countNodes());

```

21. Menampilkan isi pohon dengan tiga jenis urutan: inorder, preorder, dan postorder.

```

40         System.out.println("InOrder: ");
41         tree.printInorder();
42
43         System.out.println("\nPreOrder: ");
44         tree.printPreorder();
45
46         System.out.println("\nPostOrder: ");
47         tree.printPostorder();

```

22. Mencetak bentuk pohon secara vertikal menggunakan garis bercabang.

```

49         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
50         tree.print();

```

23. Buat file class baru dengan nama **GraphTraversal**, nantinya class ini akan kita gunakan untuk menjalankan metode pencarian traversal untuk graf seperti bfs dan dfs.

24. Deklarasikan package yang digunakan dan juga deklarasikan kelas GraphTraversal, juga deklarasikan struktur data untuk menyimpan graf berbasis adjacency list, setiap simpul punya daftar tetangganya.

```
1 package pekan9;
2 // Jovantri Immanuel Gulo - 2411532014
3 import java.util.*;
4
5 public class GraphTraversal {
6     private Map<String, List<String>> graph = new HashMap<>();
7     // Menambahkan edge (graf tak berarah)
```

25. Menambahkan sisi ke dalam graf. Karena graf tak berarah, kedua node saling ditambahkan ke daftar tetangga satu sama lain.

```
8     public void addEdge(String node1, String node2) {
9         graph.putIfAbsent(node1, new ArrayList<>());
10        graph.putIfAbsent(node2, new ArrayList<>());
11        graph.get(node1).add(node2);
12        graph.get(node2).add(node1);
13    }
```

26. Menampilkan seluruh isi graf (adjacency list), sehingga kita tahu hubungan antar simpul.

```
14    // Menampilkan graf awal
15    public void printGraph() {
16        System.out.println("Graf Awal (Adjacency List):");
17        for (String node : graph.keySet()) {
18            System.out.print(node + " → ");
19            List<String> neighbors = graph.get(node);
20            System.out.println(String.join(", ", neighbors));
21        }
22        System.out.println();
23    }
```

27. Melakukan traversal **Depth First Search** dari simpul awal secara rekursif, menggunakan dfsHelper.

```
24    // DFS rekursif
25    public void dfs(String start) {
26        Set<String> visited = new HashSet<>();
27        System.out.println("Penelusuran DFS:");
28        dfsHelper(start, visited);
29        System.out.println();
30    }
```

28. Method bantu DFS: mencetak simpul, menandai yang sudah dikunjungi, lalu memanggil dirinya sendiri ke tetangga yang belum dikunjungi.

```
32    private void dfsHelper(String current, Set<String> visited) {
33        if (visited.contains(current)) return;
34        visited.add(current);
35        System.out.print(current + " ");
36        for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
37            dfsHelper(neighbor, visited);
38        }
39    }
```

29. Melakukan traversal **Breadth First Search** secara iteratif (menggunakan queue), mencetak setiap simpul dari yang terdekat ke yang lebih jauh.

```

41 // BFS iteratif
42 public void bfs(String start) {
43     Set<String> visited = new HashSet<>();
44     Queue<String> queue = new LinkedList<>();
45     queue.add(start);
46     visited.add(start);
47     System.out.println("Penelusuran BFS:");
48     while (!queue.isEmpty()) {
49         String current = queue.poll();
50         System.out.print(current + " ");
51         for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
52             if (!visited.contains(neighbor)) {
53                 queue.add(neighbor);
54                 visited.add(neighbor);
55             }
56         }
57     }
58     System.out.println();
59 }

```

30. Pada file main java, program membuat objek graf, menambahkan sisi-sisinya (A-B, A-C, B-D, B-E), lalu mencetak graf, dan memanggil DFS serta BFS dari simpul A.

```

61 // Main
62 public static void main(String[] args) {
63     GraphTraversal graph = new GraphTraversal();
64
65     // Contoh graf: A-B, A-C, B-D, B-E
66     graph.addEdge("A", "B");
67     graph.addEdge("A", "C");
68     graph.addEdge("B", "D");
69     graph.addEdge("B", "E");
70
71     // Cetak graf awal
72     System.out.println("Graf Awal adalah: ");
73     graph.printGraph();
74
75     // Lakukan penelusuran
76     graph.dfs("A");
77     graph.bfs("A");
78 }

```

D. Kesimpulan

Dari praktikum yang telah dilakukan, dapat diambil kesimpulan bahwa pohon biner dan graf memiliki peran yang penting dalam melakukan pengolahan dan representasi data yang cukup kompleks. Implementasi pohon biner membuka kemungkinan untuk melakukan penyimpanan struktur data secara hierarki (bertingkat) serta mendukung berbagai macam traversal postorder, preorder, dan inorder. Sedangkan, untuk implementasi pohon graf dengan adanya depth first search (dfs) dan breadth first search (bfs) memungkinkan untuk melakukan eksplorasi pada seluruh simpul graf dengan tujuan dan pendekatan yang berbeda-beda.